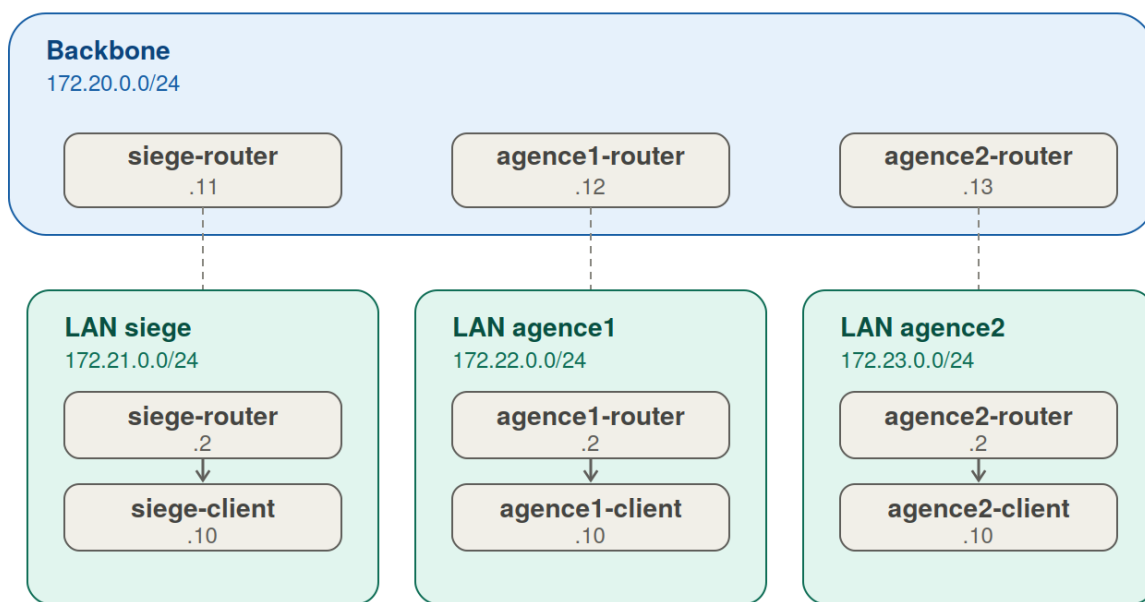


TP13 — Déployer une topologie réseau conteneurisée

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

Description



Trait pointille = meme conteneur routeur, branche sur les deux reseaux

Figure 1: Topologie réseau conteneurisée à déployer

Ce TP fait évoluer le lab Docker (TP4, un seul réseau plat) vers une vraie topologie réseau conteneurisée : chaque site a son propre réseau LAN isolé, plus un réseau “backbone” partagé qui relie les 3 sites entre eux — reproduisant en Docker la logique hub-and-spoke vue en introduction.

Prérequis

- TP1 à TP12 validés, Docker Desktop fonctionnel

Étapes

Étape 1 — Créer sa branche

```
cd netops-project-<nom-groupe>
git checkout dev
git pull origin dev
git checkout -b <prenom>-<nom>-jour4-tp13
```

Étape 2 — Arrêter l'ancien lab (topologie à plat du TP4)

```
cd lab
docker compose down
cd ..
```

Étape 3 — Réécrire docker-compose.yml avec 4 réseaux Docker

```
cat > lab/docker-compose.yml << 'EOF'
networks:
  nexacorp-backbone:
    driver: bridge
    ipam:
      config:
        - subnet: 172.20.0.0/24
  lan-siege:
    driver: bridge
    ipam:
      config:
        - subnet: 172.21.0.0/24
  lan-agence1:
    driver: bridge
    ipam:
      config:
        - subnet: 172.22.0.0/24
  lan-agence2:
    driver: bridge
    ipam:
      config:
        - subnet: 172.23.0.0/24

services:
  siege-router:
    build:
      context: ..
      dockerfile: lab/Dockerfile
    container_name: siege-router
    environment:
      - SITE_NAME=siege
```

```

    - API_TOKEN=${API_TOKEN}
ports:
  - "5001:5000"
networks:
  nexacorp-backbone:
    ipv4_address: 172.20.0.11
  lan-siege:
    ipv4_address: 172.21.0.2

siege-client:
  image: alpine:3.20
  container_name: siege-client
  command: sleep infinity
  networks:
    lan-siege:
      ipv4_address: 172.21.0.10

agencel-router:
  build:
    context: ..
    dockerfile: lab/Dockerfile
  container_name: agencel-router
  environment:
    - SITE_NAME=agencel
    - API_TOKEN=${API_TOKEN}
  ports:
    - "5002:5000"
  networks:
    nexacorp-backbone:
      ipv4_address: 172.20.0.12
    lan-agencel:
      ipv4_address: 172.22.0.2

agencel-client:
  image: alpine:3.20
  container_name: agencel-client
  command: sleep infinity
  networks:
    lan-agencel:
      ipv4_address: 172.22.0.10

agence2-router:
  build:
    context: ..
    dockerfile: lab/Dockerfile
  container_name: agence2-router
  environment:
    - SITE_NAME=agence2

```

```

    - API_TOKEN=${API_TOKEN}
ports:
  - "5003:5000"
networks:
  nexacorp-backbone:
    ipv4_address: 172.20.0.13
  lan-agence2:
    ipv4_address: 172.23.0.2

agence2-client:
  image: alpine:3.20
  container_name: agence2-client
  command: sleep infinity
  networks:
    lan-agence2:
      ipv4_address: 172.23.0.10
EOF

```

Chaque routeur (le mock Flask) est branché sur deux réseaux : son LAN local (backbone .2, voir note ci-dessous) et le backbone partagé — un conteneur “multi-homed”, comme un vrai routeur avec une interface LAN et une interface WAN. Chaque client (une machine Alpine minimaliste) est branché uniquement sur le LAN de son site.

Pourquoi .2 et pas .1 pour les routeurs sur leur LAN : sur un réseau bridge Docker, l’adresse .1 du sous-réseau est automatiquement réservée par Docker comme adresse de la passerelle (gateway) du réseau. L’assigner à un conteneur provoque `Error response from daemon: failed to set up container networking: Address already in use.`

Étape 4 — Ajouter les outils réseau à l’image (ping, curl)

L’image `python:3.12-slim` utilisée par nos routeurs est minimaliste et n’inclut ni ping ni curl — nécessaires pour les tests de connectivité de ce TP.

```

sed -i 's/RUN pip install --no-cache-dir flask/a RUN apt-get update \&\& apt-get install -y --no-install-recommends iputils-ping curl \&\& rm -rf /var/lib/apt/lists/*' lab/Dockerfile
cat lab/Dockerfile

```

Étape 5 — Démarrer la nouvelle topologie

```

cd lab
docker compose up -d --build
docker compose ps
cd ..

```

Résultat attendu : 6 conteneurs démarrés (3 routeurs + 3 clients), sans erreur.

Étape 6 — Visualiser les réseaux créés

```

docker network ls | grep lab

```

Résultat attendu : 4 réseaux (lab_nexacorp-backbone, lab_lan-siege, lab_lan-agence1, lab_lan-agence2).

```
docker network inspect lab_nexacorp-backbone --format '{{range .Containers}}{{.Name}} {{.IPv4Address}}{{"\n"}}{{end}}'
```

Résultat attendu : les 3 routeurs, chacun avec son IP sur le backbone (172.20.0.11, .12, .13).

Étape 7 — Vérifier qu'un client atteint son propre routeur (même LAN)

```
docker exec siege-client ping -c 2 172.21.0.2
```

Résultat attendu : 2 paquets reçus, 0% de perte.

Étape 8 — Vérifier l'isolation entre LAN de sites différents

```
docker exec siege-client ping -c 2 172.22.0.10
```

Résultat attendu : échec (Destination Host Unreachable ou timeout) — siege-client ne peut pas atteindre agence1-client, ils sont sur des réseaux Docker complètement séparés, sans route entre eux.

Étape 9 — Vérifier que les routeurs, eux, communiquent via le backbone

```
docker exec siege-router ping -c 2 172.20.0.12
```

```
docker exec siege-router curl -s http://172.20.0.12:5000/health
```

Résultat attendu : ping réussi, et la requête curl retourne {"site": "agence1", "status": "ok"} — les routeurs se voient entre eux via le backbone, alors que leurs LAN respectifs restent cloisonnés.

Étape 10 — Vérifier que l'accès externe fonctionne toujours

```
curl -s http://localhost:5001/health
```

```
curl -s http://localhost:5002/health
```

```
curl -s http://localhost:5003/health
```

Résultat attendu : les 3 sites répondent normalement — le mapping de ports fonctionne indépendamment de la segmentation interne.

Étape 11 — Relancer les outils des TP précédents pour confirmer que rien n'est cassé

```
export VAULT_API_TOKEN=s3cr3t-token-nexacorp
```

```
python scripts/run_diagnostics.py
```

Résultat attendu : les 3 sites toujours diagnostiqués correctement.

Étape 12 — Committer et pousser

```
git add lab/docker-compose.yml lab/Dockerfile
git commit -m "TP13 : topologie reseau conteneurisee (backbone + LAN isolés par site)"
git push -u origin <prenom>-<nom>-jour4-tp13
```

Ouvrir une pull request <prenom>-<nom>-jour4-tp13 → dev.

Comprendre le TP

Un réseau Docker est un vrai domaine de diffusion isolé : deux conteneurs sur des réseaux Docker différents ne peuvent pas se parler directement, même sur la même machine — Docker crée des bridges Linux distincts, pas juste une étiquette. C’est ce qui explique l’échec de l’étape 8 : `siege-client` et `agence1-client` n’ont tout simplement aucune route l’un vers l’autre.

Un conteneur “multi-homed” (branché sur plusieurs réseaux à la fois, comme nos routeurs) joue le rôle d’un vrai routeur : une patte sur le LAN local, une patte sur le backbone partagé — c’est ce qui permettrait, dans un scénario plus poussé, à un client du LAN d’atteindre un autre site via son routeur.

L’adresse .1 réservée par Docker est un piège classique : sur tout réseau bridge Docker, cette adresse appartient à la passerelle du réseau — jamais assignable à un conteneur.

ipv4_address fixe rend les adresses prévisibles d’une exécution à l’autre, utile en lab pédagogique — en production, on utiliserait plutôt la résolution DNS interne de Docker (chaque conteneur joignable par son nom de service).

Validation

- ☐ `docker compose ps` montre 6 conteneurs actifs (3 routeurs + 3 clients)
- ☐ `docker network ls` montre 4 réseaux distincts
- ☐ Un client atteint son propre routeur (même LAN)
- ☐ Un client n’atteint pas le client d’un autre site (isolation confirmée)
- ☐ Les routeurs se joignent entre eux via le backbone (`ping` + `curl /health` réussis)
- ☐ L’accès externe (`curl localhost:500x/health`) fonctionne toujours
- ☐ Le script de diagnostic (TP12) fonctionne toujours sans modification
- ☐ Une pull request <prenom>-<nom>-jour4-tp13 → dev est ouverte